How The Apps Work Together:

A) Simplified to two main apps, logical grouping of related features
B) Users app handles authentication, profiles and user related functionalities
C) Projects app handles everything video-related

1.”users” App (User Management) (***Key Models-User, Profile, Follow***)

Core Purpose -	Advanced Purpose -
1.Handles user accounts (signup/login)✔️ 2.Manages user profiles✔️ 3.Stores user information✔️ 4.Manages the ‘user to user following’ feature✔️ 5.Handles Authentication & Authorization✔️ 6.Core Authen. & Auth. Security (URL, etc) ❓	1.Monetization ⁉️⁉️

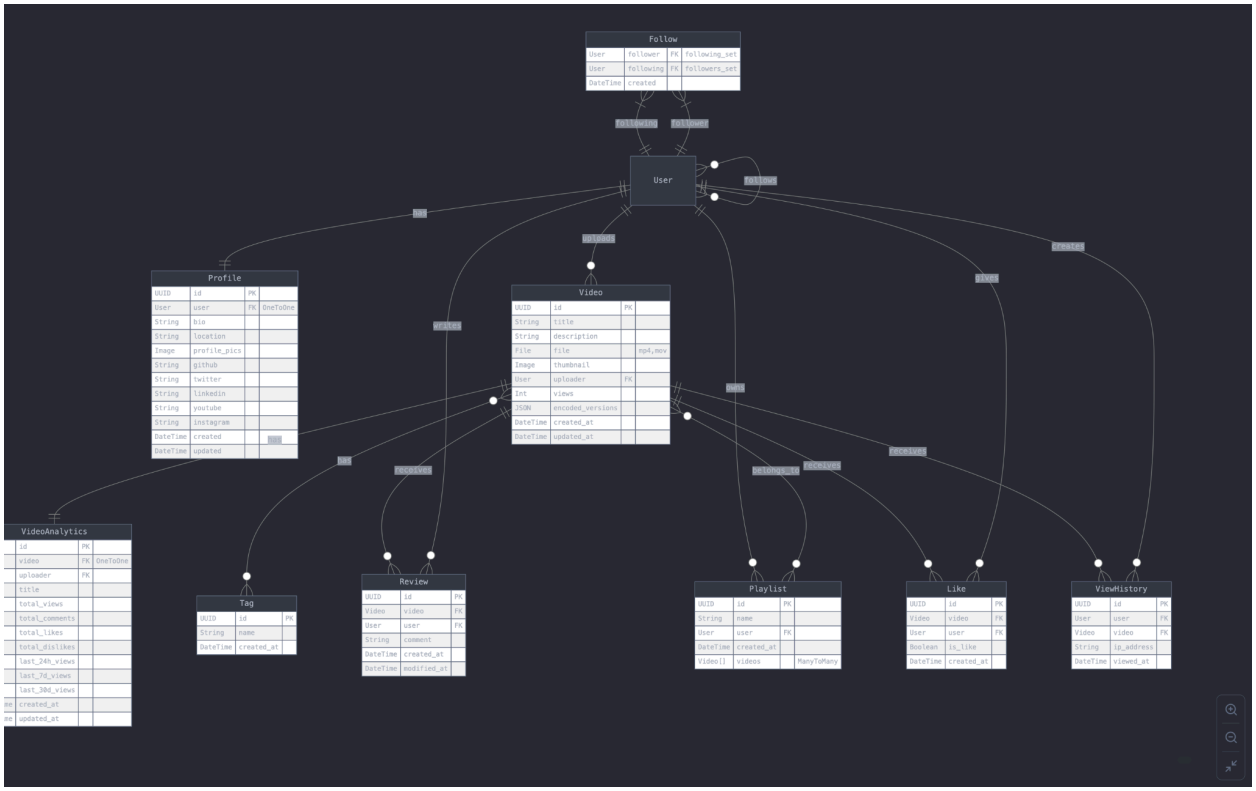
2.”projects” App (Core Features) (***Key Models-Video, Tag, Like, Review, Playlist, ViewHistory, VideoAnalytics)***

Core Purpose -	Advanced Purpose -
1.Handles video uploads ✔️ 2.Manages video playback ✔️ 3.Stores all the relevant video metadata ❓ 4.Manages video listings ✔️ 5.Handles video reviews/ratings ✔️ 6.Manages video tagging system ✔️ 7.Fitting the thumbnails with Bars ✔️ 8.Managing user’s viewing history ✔️ 9.Video Compression and Encoding ✔️ 10. Video analytics page ✔️ 11.Task queues for video processing ❓ ❓ 12.Better caching strategy ❓ ❓ 13.CDN integration ❓ ❓ 14.Load balancing ❓ ❓ 15.Dynamic Comment Section ✔️ 16. Database Sharding ❓ ❓ 17.Unique Video Recs on Homepage ❓	1.Thumbnail Generation ⁉️ 2.Ability to Handle Live Streaming ⁉️⁉️ 3.Security and Compliance (Copyrite) ⁉️⁉️ 4.Creator Tools Page ⁉️⁉️

How to Transition Your Code to Industry Standards

1. Separate concerns (HTML, CSS, JS).
2. Adopt modern JavaScript features like fetch, async/await, and template literals.
3. Error handling: **Make sure every interaction has proper success/failure feedback.**
4. Security: **Validate and sanitize inputs.**
5. Responsive design: Use CSS classes to control UI behavior instead of inline styles.
6. **Document everything important** in the codebase

DB Schema



Key Relationships Between Tables:

1) User - Profile Relationship Practical: Each user has exactly one profile, and each profile belongs to exactly one user Technical: One-to-One relationship with Profile.user_id referencing User.id Example: Like a YouTube account (User) having one channel profile (Profile) Access: user.profile or profile.user	2) User - Video Relationship Practical: One user can upload many videos, but each video has one uploader Technical: One-to-Many relationship with Video.uploader_id referencing User.id Example: A YouTuber uploading multiple videos to their channel Access: user.videos.all() or video.uploader	3) User - Review Relationship Practical: One user can write many reviews, each review has one author Technical: One-to-Many relationship with Review.user_id referencing User.id Example: A user commenting on multiple different videos Access: user.reviews.all() or review.user

4) User - Like Relationship Practical: One user can like many videos, each like comes from one user Technical: One-to-Many with Like.user_id referencing User.id Example: A user liking multiple different videos Access: user.video_likes.all() or like.user	5) User - Playlist Relationship Practical: One user can create many playlists, each playlist has one owner Technical: One-to-Many with Playlist.user_id referencing User.id Example: A user creating multiple playlists for different video categories Access: user.playlists.all() or playlist.user	6) Video - Tag Relationship Practical: One video can have many tags, one tag can be on many videos Technical: Many-to-Many relationship using an automatic junction table Example: A video having multiple tags (#funny, #music, #tutorial) Access: video.tags.all() or tag.video_set.all()
7) Video - Playlist Relationship Practical: One video can be in many playlists, one playlist can have many videos Technical: Many-to-Many relationship using an automatic junction table Example: Adding a music video to multiple different playlists Access: playlist.videos.all() or video.playlists.all()	8) User - Follow Relationship Practical: Users can follow many users and be followed by many users Technical: Many-to-Many through explicit Follow model with additional data Example: Like following multiple YouTube channels Access: user.following_set.all() or user.followers_set.all()	9) Video - Like Relationship Practical: One video can have many likes, each like belongs to one video Technical: One-to-Many with Like.video_id referencing Video.id Example: Multiple users liking or disliking a single video Access: video.likes.all() or like.video
10) Video - Review Relationship Practical: One video can have many comments, each comment belongs to one video Technical: One-to-Many with Review.video_id referencing Video.id Example: Like YouTube comments where multiple users can comment on a single video	11) Video - ViewHistory Relationship Practical: One video can have many view records Technical: One-to-Many with ViewHistory.video_id referencing Video.id Example: Tracking individual views of a video Access: video.views_history.all() Features: Tracks timestamp and IP for anonymous views	12) User - ViewHistory Relationship Practical: Users can have viewing history for multiple videos (nullable) Technical: One-to-Many with ViewHistory.user_id referencing User.id (null=True) Example: Tracking what videos a user has watched Access: user.view_history.all()

13) Video - VideoAnalytics Relationship Practical: Each video has exactly one analytics dashboard tracking engagement metrics Technical: One-to-One relationship with VideoAnalytics.video_id referencing Video.id Example: Like a YouTube Studio analytics panel showing comprehensive video statistics	14) User - VideoAnalytics Relationship Practical: A user can see analytics for all their uploaded videos Technical: One-to-Many with VideoAnalytics.uploader_id referencing User.id Example: A creator viewing analytics across their entire video catalog	

GITHUB WORKFLOW

1.Initial Status and Feature Branch Work	2. Update Main Branch	3.Update Develop Branch
# Check current branches (*indicates current branch) > git branch # Stage all changes > git add . # Commit changes with message > git commit -m "your message" # Push feature branch to remote > git push origin feature/branch-name	# Switch to main branch > git checkout main # Pull changes from develop to main > git pull origin develop # Push changes to main > git push origin main #If rejected: # sync with remote main > git pull origin main # try pushing again > git push origin main	# Switch to develop > git checkout develop # Get latest changes > git pull origin develop # Merge feature branch into develop > git merge feature/branch-name # Push updated develop > git push origin develop
4.Clean Up Feature Branch	5.Start New Feature Branch	6.Final Sync
# Delete local feature branch > git branch -d feature/branch-name # Delete remote feature branch > git push origin --delete feature/branch-name	# Create and switch to new feature branch from develop > git checkout develop > git checkout -b feature/new-branch-name # Push new branch to remote and set tracking > git push -u origin feature/new-branch-name	# Get all remote updates > git fetch --all # Pull updates for all branches > git pull --all # Verify branch structure > git branch

Video Upload Pipeline

In our advanced video processing implementation, we've developed a sophisticated pipeline that handles the journey of video content from upload through processing to final storage and delivery. The system represents a careful balance between processing efficiency, storage optimization, and delivery performance, utilizing modern compression techniques and intelligent data management strategies.

The journey begins when a user initiates a video upload through our custom-built ProjectForm interface. This form, beyond merely accepting file uploads, implements a comprehensive validation layer that ensures incoming videos meet our platform's specifications. The system accepts MP4 and MOV formats, immediately validating these through Django's FileExtensionValidator before any processing begins. This initial validation layer serves as our first defense against invalid uploads and potential system abuse.

Upon successful validation, our create_project view initiates a carefully orchestrated processing sequence. The system generates a unique submission hash using MD5 encryption, preventing duplicate uploads through an intelligent caching mechanism. This hash serves a dual purpose: preventing redundant processing while also providing a reliable identifier for tracking the video throughout its processing lifecycle. The implementation wraps this entire process in a database transaction, ensuring atomic operations and maintaining data consistency even in case of processing failures.

The core of our processing pipeline lies in the VideoService's handle_video_save method, which orchestrates the complex interplay between temporary storage, processing, and final persistence. When a new video is detected through our file_changed flag system, it triggers our FFmpeg-based processing pipeline. This pipeline, implemented in our encode_video function, represents a sophisticated approach to video optimization. It begins with a detailed probe of the video file, analyzing its characteristics to determine optimal processing parameters.

Our compression and encoding strategy implements a multi-tiered approach to quality optimization. The system utilizes the H.264 codec with a carefully calibrated Constant Rate Factor of 28, striking an optimal balance between quality retention and file size reduction. The pipeline implements sophisticated bitrate management with a 2M maxrate and 4M buffer size, ensuring consistent quality across varying content complexity. Audio processing isn't neglected, with AAC encoding at 96k providing excellent audio quality while minimizing file size impact.

The system's intelligence extends to its quality version management. Rather than a one-size-fits-all approach, our pipeline generates multiple encoded versions optimized for different delivery scenarios. The primary 720p version serves as our standard quality tier, while additional versions can be generated based on platform requirements. Each version undergoes careful optimization, with parameters tuned for specific resolution targets while maintaining visual fidelity.

Storage management represents another sophisticated aspect of our implementation. The ModelService handles the cleanup of original files through an intelligent verification system that ensures encoded versions are successfully created before removing source files. This approach optimizes storage utilization while maintaining system reliability. The video's metadata, including references to all encoded versions, is stored in a JSONField, providing flexible schema evolution while maintaining query efficiency.

Error handling and logging permeate every level of our implementation. The system implements comprehensive error catching and logging strategies, ensuring that processing failures are properly documented and can be analyzed for system improvement. This robust error handling extends to our transaction management, ensuring database consistency even in the face of processing failures.

The final product of this pipeline is a fully optimized video entry in our database, complete with multiple encoded versions suitable for various delivery scenarios. Our implementation of streaming delivery utilizes HTTP range requests and chunked transfer encoding, ensuring efficient delivery while minimizing server resource utilization. The system's caching strategy, implemented through Django's cache framework, ensures frequently accessed videos are served with minimal processing overhead.

This entire system represents a careful balance between processing thoroughness and system performance. By implementing intelligent queuing and background processing, we ensure that resource-intensive operations don't impact system responsiveness. The result is a robust, scalable video processing pipeline capable of handling significant upload volumes while maintaining consistent processing quality and system performance.

Video Streaming Implementation Process

Our video streaming implementation follows a sophisticated process that efficiently delivers video content from server storage to end users. The process begins when a user requests a video through our web interface. Instead of loading the entire video file into memory, as was done in our previous implementation, we now utilize a chunked streaming approach.
When the video player initializes, it first requests only the video metadata. This metadata includes information about the video duration, dimensions, and available quality versions. The server, through our VideoService, determines the appropriate version of the video to serve, typically defaulting to 720p if available, with fallback options to other qualities or the original file if necessary.

As playback begins, the video player sends HTTP range requests to our server. These requests specify exactly which portions of the video file are needed at any given moment. Our stream_video view function processes these requests, using the file_iterator to read the video file in small, manageable chunks of 8KB (8192 bytes). This chunked approach means that only a small portion of the video is held in memory at any given time, significantly reducing server resource usage.

The file_iterator function is central to our streaming process. When it receives a range request, it opens the video file, seeks to the specified position, and yields data in controlled chunks. This streaming process continues throughout the video playback, with the player requesting new chunks as needed. For seeking operations, when a user jumps to a different part of the video, the player simply sends a new range request for that specific portion of the file.

Our implementation includes built-in support for quality management through encoded versions of each video. The ModelService handles the URL generation for these different quality versions, while the VideoService manages the overall video delivery process, including view counting and caching of video context data. This service-oriented architecture ensures clean separation of concerns and maintainable code.

The actual delivery of video content to the client is handled through Django's StreamingHttpResponse, which maintains an open connection to stream the video chunks. The response includes proper HTTP headers for content range and length, allowing the client's video player to properly handle the incoming stream. This streaming response is crucial for efficient memory usage and proper handling of large video files.

On the frontend, we utilize the Video.js player, which is configured to handle these chunked streams efficiently. The player maintains its own buffer of video data, requesting new chunks as needed based on the user's playback position and network conditions. Our custom CSS ensures proper video display regardless of the video's original dimensions, maintaining aspect ratios while providing a consistent user interface.

This entire process represents a significant improvement over traditional video serving methods. Where previously we would load entire video files into memory, potentially causing server strain with multiple concurrent users, we now handle video delivery in a memory-efficient, scalable manner. The chunked streaming approach, combined with quality management and proper range request handling, provides a smooth, efficient video playback experience while maintaining server stability under load.

The end result is a robust video streaming system that efficiently manages server resources while providing users with smooth, high-quality video playback. The system handles everything from initial video request through continuous playback and seeking operations, all while maintaining minimal memory footprint and optimal performance characteristics.

Dynamic Comment Section

The dynamic comments system represents a sophisticated approach to real-time user interaction in our video platform. Unlike traditional comment systems that require page refreshes, our implementation provides a seamless, single-page application experience while maintaining data consistency and user engagement.

When a user first loads a video page, the system initializes by embedding existing comments directly into the page's HTML. This initial state is cleverly stored in a hidden input element as JSON data, allowing immediate access to comments without additional server requests. This approach significantly reduces initial load time and provides a smooth user experience from the moment the page loads.

The heart of our commenting system lies in its event handling architecture. Rather than attaching individual event listeners to each button or element, we implement a central event delegation system. This system captures all click events at the document level and determines the appropriate action based on the clicked element. This approach not only improves performance but also handles dynamically added content without requiring additional event binding.

Comment creation follows a sophisticated process. When a user clicks the "Add Comment" button, the system smoothly reveals a comment form. As the user types and submits their comment, our JavaScript handler intercepts the form submission, preventing the default page reload. Instead, it packages the form data and sends it to the server via an AJAX request. The server processes this request, stores the comment in the database, and returns the formatted comment data. The frontend then seamlessly inserts this new comment into the page's DOM, providing immediate visual feedback to the user.

The editing and deletion processes follow similarly sophisticated patterns. For comment owners, edit and delete buttons appear alongside their comments. Clicking edit transforms the comment text into an editable form, while delete triggers a confirmation modal. Both actions communicate with the server asynchronously, updating the UI only after successful server responses. This ensures data consistency while maintaining a responsive user interface.

Our backend architecture centers around the Review model, which stores essential comment data including the comment text, user reference, video reference, and timestamps. Each comment is assigned a unique UUID, ensuring reliable identification across the system. The model's structure supports efficient querying and maintains proper relationships between users, videos, and comments.

The server-side implementation includes robust view functions that handle comment creation, updates, and deletion. These views incorporate proper authentication checks, ensuring that users can only modify their own comments. They return JSON responses that the frontend can easily process and display. The system also includes protection against common issues like double submissions and race conditions.

The frontend JavaScript code is organized around clear responsibilities. The createCommentRow function handles the HTML generation for comments, ensuring consistent formatting. The handleSubmit function manages form submissions, including preventing double submissions and providing user feedback. The system also includes sophisticated error handling, displaying appropriate messages to users when issues occur.

This entire system works together to provide a modern, responsive commenting experience. Users can interact with comments instantly, seeing their changes reflected immediately while the system ensures all data is properly synchronized with the server. The implementation balances performance, user experience, and data integrity, creating a robust platform for user engagement through comments.

The result is a commenting system that feels natural and responsive, encouraging user interaction while maintaining data consistency and system reliability. Through careful architecture and modern web technologies, we've created a seamless experience that enhances the overall video platform functionality.